

USER_WORKFLOW

Mercatus Arena - User Workflow Guide Summary

TechVerse 2026 Algorithmic Trading Evaluation Platform

1. Overview

Mercatus Arena is a live algorithmic trading simulator where teams compete by running automated trading bots against a synthetic market feed. Prices update every second via WebSocket, and the event runs for a fixed duration with a designated blackout period.

User Roles

- **Team:** Trades via an automated bot or manual console, views the leaderboard, and submits code and strategy reports.
 - **Admin:** Starts and manipulates the event, manages accounts, and handles overall scoring.
 - **Evaluator (Judge):** Reviews team submissions, assigning scores for code quality and strategy reports.
-

2. Event Lifecycle

The event simulation progresses through distinct phases:

1. **PRE_LAUNCH:** Teams register and access the dashboard. Trading is disabled, and API keys remain masked.
 2. **ACTIVE_MARKET:** The market opens, API keys are revealed, and teams are reset to their starting capital. Prices stream live, and bots execute trades.
 3. **ACTIVE_MARKET (Frozen):** Typically around T-20 minutes to the end, the leaderboard takes a final snapshot and freezes, hiding live rankings.
 4. **API_FROZEN:** At T-15 minutes, the API freezes. All buy and sell orders are rejected, but submissions (code and reports) are still accepted.
 5. **EVENT_CONCLUDED:** The event finishes, final rankings are computed, and composite scores are finalized.
-

3. Team Registration & Dashboard View

- **Registration & Login:** Teams register with a unique name (2-64 characters) and password. Upon login, teams are redirected to their Dashboard.
 - **The Trading Desk (Dashboard):** Features live market charts, real-time portfolio tracking (updated every 3 seconds), an order entry panel, a mini leaderboard, open positions tracking, and a submission panel for uploading reports and repository links.
 - **Candlestick Charts:** Powered by TradingView, these charts support various timeframes ranging from 1-second ultra-high-frequency ticks to 1-hour overviews for long-term trends.
-

4. Trading Mechanics & Rules

- **Placing Orders:** Orders can be placed manually via the console or automatically via API. Users must specify the symbol, quantity, and an optional limit price.
 - **Core Rules & Constraints:**
 - **Long-only:** No shorting is allowed. Selling more shares than currently held will result in rejection.
 - **Capital & Fees:** All teams begin with identical starting capital (default ₹10,000,000). There are zero trading fees and no slippage; market orders fill exactly at the live price.
 - **Order Constraints:** Quantities must be integers between 1 and 100,000,000 per order.
 - **Rate Limits:** API limits are strictly enforced (e.g., 300 req/min for overall API, 60 req/min for trading).
-

5. AlgoBot (Automated Trading)

Teams are provided with a base Python trading bot (`my_algo.py`).

- **Execution:** Run via the CLI using `python3 my_algo.py --minutes <runtime>`.
- **Configuration:** Accepts parameters such as smoothing factors (`--alpha`), dip thresholds (`--entry-z`), stop-loss thresholds (`--stop-z`), and risk management settings (`--risk` , `--max-exposure`).
- **Strategies Used:**
 - *Momentum (CORE):* Buys upon uptrend confirmation and holds until reversal.
 - *Mean-reversion (SCALP):* Capitalizes on noise dips and threshold pops.
 - *Flash plays:* Executes on extreme one-tick dips.
- **Dry Run:** Teams can test their configurations safely using the `--dry` flag, which prints trade signals without executing them on the server.

6. Submissions & API Reference

Submitting Code & Report

- Submissions require a GitHub/GitLab repository link and a Strategy Report PDF.
- **Constraints:** The PDF must be under 10 MB and begin with standard `%PDF-` magic bytes. Submissions can be updated iteratively until the event concludes.

Key API Endpoints

- `GET /api/market/status` : Fetch market state and configuration.
- `POST /api/trade/buy & /sell` : Execute orders (requires API Key).
- `GET /api/team/portfolio` : Retrieve cash, holdings, and PnL metrics.
- **WebSocket (`ws://localhost:8080/ws`)**: Provides live streaming for market ticks and execution updates.

7. Administration & Evaluation

Admin Console

Admins have granular control over the simulation:

- **Event Control:** Start, pause, resume, or conclude the event; reveal API keys.
- **Market Manipulation:** Dynamically adjust price volatility, replay speed, or inject sudden flash crash shocks (-30% to +30%).
- **Data Management:** Upload custom pricing datasets via CSV (up to 200 MB) or generate synthetic ones on the fly.

Evaluator (Judge) Desk

Evaluators have access to a comprehensive audit trail:

- **Review Process:** Evaluators can inspect each team's trade ledger, view individual execution latencies, check HTTP request logs, and read strategy reports.
- **Scoring:** Evaluators assign a Code Quality score (0-100) and a Strategy Report score (0-100). LLM autograding is also supported for automatic preliminary scoring.

8. Scoring & Final Results

The final standings are calculated using a weighted composite formula:

Final Score = (PnL Rank Score × 0.50) + (Code Quality × 0.25) + (Strategy Report × 0.25)

- *PnL Rank Score*: Based directly on leaderboard position.
 - *Code Quality & Strategy Report*: Based on Judge/LLM evaluations.
-

9. Quick Start Checklist

1. Register your team at `/register`.
2. Wait for the admin to officially start the event and unmask API credentials.
3. Retrieve your API key from the Dashboard.
4. Export your API key locally and configure AlgoBot.
5. Launch your bot (e.g., `python3 my_algo.py --minutes 60`).
6. Monitor live portfolio metrics and system limits on the Dashboard.
7. Submit your repository link and strategy PDF prior to the event conclusion.
8. Review the final rankings on the Leaderboard once scoring computation finishes.